

**Министерство науки и высшего образования Российской Федерации**  
**Федеральное государственное автономное образовательное учреждение**  
**высшего образования**  
**«Пермский национальный исследовательский политехнический**  
**университет»**

Электротехнический факультет

Кафедра: Информационные технологии и автоматизированные системы  
(ИТАС)

Направление подготовки: 09.03.04 – «Программная инженерия»

**ОТЧЕТ**

**Лабораторная работа №13.**

**Тема: «Стандартные обобщенные алгоритмы библиотеки STL»**

**По дисциплине «Основы алгоритмизации и программирования»**

**Вариант 13**

Выполнила: студентка группы РИС-25-26

Лаптева Елена Владимировна

Принял: доц. Полякова О.А.

Пермь, 2026

## Постановка задачи:

|    |                                                                                                                                                                                                                             |                                                                          |                                                                |
|----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------|----------------------------------------------------------------|
| 13 | <b>Задача 1</b><br>1. Контейнер - двунаправленная очередь<br>2. Тип элементов Pair (см. лабораторную работу №3).<br><b>Задача 2</b><br>Адаптер контейнера – стек.<br><b>Задача 3</b><br>Ассоциативный контейнер - множество |                                                                          |                                                                |
|    | <b>Задание 3</b>                                                                                                                                                                                                            | <b>Задание 4</b>                                                         | <b>Задание 5</b>                                               |
|    | Найти максимальный элемент и добавить его в конец контейнера                                                                                                                                                                | Найти элементы ключами из заданного диапазона и удалить их из контейнера | К каждому элементу добавить среднее арифметическое контейнера. |

## Анализ задачи:

### Задача 1.

1. Создаем последовательный контейнер.
2. Заполняем его элементами пользовательского типа. Для пользовательского типа перегружаем необходимые операции.
3. Заменяем элементы в соответствии с заданием (используем алгоритмы `replace_if()`, `replace_copy()`, `replace_copy_if()`, `fill()`).
4. Удаляем элементы в соответствии с заданием (используем алгоритмы `remove()`, `remove_if()`, `remove_copy_if()`, `remove_copy()`).
5. Сортируем контейнер по убыванию и по возрастанию ключевого поля (используем алгоритм `sort()`).
6. Находим в контейнере заданный элемент (используем алгоритмы `find()`, `find_if()`, `count()`, `count_if()`).

### Задача 2.

1. Создаем адаптер контейнера.
2. Заполняем его элементами пользовательского типа. Для пользовательского типа перегружаем необходимые операции.
3. Заменяем элементы в соответствии с заданием (используем алгоритмы `replace_if()`, `replace_copy()`, `replace_copy_if()`, `fill()`).
4. Удаляем элементы в соответствии с заданием (используем алгоритмы `remove()`, `remove_if()`, `remove_copy_if()`, `remove_copy()`).

5. Сортируем контейнер по убыванию и по возрастанию ключевого поля (используем алгоритм `sort()`).

6. Находим в контейнере элемент с заданным ключевым полем (используем алгоритмы `find()`, `find_if()`, `count()`, `count_if()`).

### Задача 3

1. Создаем ассоциативный контейнер.

2. Заполняем его элементами пользовательского типа. Для пользовательского типа перегружаем необходимые операции.

3. Заменяем элементы в соответствии с заданием (используем алгоритмы `replace_if()`, `replace_copy()`, `replace_copy_if()`, `fill()`).

4. Удаляем элементы в соответствии с заданием (используем алгоритмы `remove()`, `remove_if()`, `remove_copy_if()`, `remove_copy()`).

5. Сортируем контейнер по убыванию и по возрастанию ключевого поля (используем алгоритм `sort()`).

6. Находим в контейнере элемент с заданным ключевым полем (используем алгоритмы `find()`, `find_if()`, `count()`, `count_if()`).

Код:

```
#include<iostream>

#include<deque>

#include<stack>

#include<set>

#include<vector>

#include<algorithm>

#include<cstdlib>

#include<ctime>

using namespace std;

// ===== КЛАСС PAIR =====

class Pair {
private:
    double first;
    double second;
public:
```

```

Pair() : first(0), second(0) {}

Pair(double f, double s) : first(f), second(s) {}

Pair(const Pair& p) : first(p.first), second(p.second) {}

~Pair() {}

```

```

double getFirst() const { return first; }

double getSecond() const { return second; }

void setFirst(double f) { first = f; }

void setSecond(double s) { second = s; }

```

```

Pair& operator=(const Pair& p) {
    if (&p == this) return *this;
    first = p.first; second = p.second;
    return *this;
}

```

```

Pair operator+(const Pair& p) const {
    return Pair(first + p.first, second + p.second);
}

```

```

Pair operator/(const double& d) const {
    return Pair(first / d, second / d);
}

```

```

bool operator>(const Pair& p) const {
    return (first > p.first) || (first == p.first && second > p.second);
}

```

```

bool operator<(const Pair& p) const {
    return (first < p.first) || (first == p.first && second < p.second);
}

```

```

bool operator>=(const Pair& p) const { return !(*this < p); }

bool operator<=(const Pair& p) const { return !(*this > p); }

bool operator==(const Pair& p) const { return first == p.first && second == p.second; }

```

```

friend istream& operator>>(istream& in, Pair& p) {
    cout << "first? "; in >> p.first;
    cout << "second? "; in >> p.second;
    return in;
}

friend ostream& operator<<(ostream& out, const Pair& p) {
    out << "(" << p.first << ", " << p.second << ")";
    return out;
}
};

//=====
// ЗАДАЧА 1: deque<Pair> с алгоритмами STL
//=====

typedef deque<Pair> Deque;

Deque make_deque(int n) {
    Deque d;
    for (int i = 0; i < n; i++)
        d.push_back(Pair(rand() % 50, rand() % 50));
    return d;
}

void print_deque(const Deque& d) {
    for (auto it = d.begin(); it != d.end(); ++it)
        cout << *it << " ";
    cout << endl;
}

// Задание 3: Найти максимальный элемент и добавить его в конец
void task1_add_max(Deque& d) {
    if (d.empty()) return;
    auto max_it = max_element(d.begin(), d.end());
    d.push_back(*max_it);
    cout << "Максимальный элемент (" << *max_it << ") добавлен в конец" << endl;
}

```

```
}
```

```
// Задание 4: Удалить элементы из диапазона (remove_if)
```

```
void task1_delete_range(Deque& d, Pair key1, Pair key2) {  
    if (key1 > key2) swap(key1, key2);  
    auto new_end = remove_if(d.begin(), d.end(),  
        [key1, key2](const Pair& val) { return val >= key1 && val <= key2; });  
    int count = distance(new_end, d.end());  
    d.erase(new_end, d.end());  
    cout << "Удалено элементов: " << count << endl;  
}
```

```
// Задание 5: Добавить среднее арифметическое (for_each)
```

```
void task1_add_average(Deque& d) {  
    if (d.empty()) return;  
    Pair sum(0, 0);  
    for_each(d.begin(), d.end(), [&sum](const Pair& val) { sum = sum + val; });  
    Pair avg = sum / d.size();  
    for_each(d.begin(), d.end(), [avg](Pair& val) { val = val + avg; });  
    cout << "Среднее арифметическое добавлено к каждому элементу" << endl;  
}
```

```
void run_task1() {  
    cout << "\n===== ЗАДАЧА 1: deque<Pair> =====" << endl;  
    Deque d;  
    int n;  
    cout << "Введите количество элементов: ";  
    cin >> n;  
    d = make_deque(n);  
    cout << "Исходный deque: ";  
    print_deque(d);  
  
    int choice;  
    cout << "\nВыберите задание:\n";  
    cout << "1. Задание 3 - Добавить максимальный элемент в конец (max_element)\n";  
    cout << "2. Задание 4 - Удалить элементы из диапазона (remove_if)\n";
```

```

cout << "3. Задание 5 - Добавить среднее арифметическое (for_each)\n";
cout << "Ваш выбор: ";
cin >> choice;

switch (choice) {
case 1:
    task1_add_max(d);
    cout << "Результат: ";
    print_deque(d);
    break;
case 2: {
    Pair key1, key2;
    cout << "Введите начало диапазона (Pair):\n";
    cin >> key1;
    cout << "Введите конец диапазона (Pair):\n";
    cin >> key2;
    task1_delete_range(d, key1, key2);
    cout << "Результат: ";
    print_deque(d);
    break;
}
case 3:
    task1_add_average(d);
    cout << "Результат: ";
    print_deque(d);
    break;
default:
    cout << "Неверный выбор!" << endl;
}
}

//=====
// ЗАДАЧА 2: stack<Pair> с алгоритмами STL
//=====

typedef stack<Pair> Stack;
typedef vector<Pair> Vector;

```

```

Vector copy_stack_to_vector(Stack s) {
    Vector v;
    while (!s.empty()) {
        v.push_back(s.top());
        s.pop();
    }
    return v;
}

```

```

Stack copy_vector_to_stack(Vector v) {
    Stack s;
    for (int i = v.size() - 1; i >= 0; i--)
        s.push(v[i]);
    return s;
}

```

```

Stack make_stack(int n) {
    Stack s;
    Pair p;
    for (int i = 0; i < n; i++) {
        cout << "Элемент " << i + 1 << ":\n";
        cin >> p;
        s.push(p);
    }
    return s;
}

```

```

void print_stack(Stack s) {
    Vector v = copy_stack_to_vector(s);
    cout << "Стек (сверху вниз): ";
    for (int i = v.size() - 1; i >= 0; i--)
        cout << v[i] << " ";
    cout << endl;
}

```



// Задание 3: Найти максимальный элемент и добавить в вершину

```
void task2_add_max(Stack& s) {  
    if (s.empty()) return;  
    Vector v = copy_stack_to_vector(s);  
    auto max_it = max_element(v.begin(), v.end());  
    s = copy_vector_to_stack(v);  
    s.push(*max_it);  
    cout << "Максимальный элемент добавлен в вершину стека" << endl;  
}
```

// Задание 4: Удалить элементы из диапазона (remove\_if)

```
void task2_delete_range(Stack& s, Pair key1, Pair key2) {  
    if (key1 > key2) swap(key1, key2);  
    Vector v = copy_stack_to_vector(s);  
    auto new_end = remove_if(v.begin(), v.end(),  
        [key1, key2](const Pair& val) { return val >= key1 && val <= key2; });  
    v.erase(new_end, v.end());  
    s = copy_vector_to_stack(v);  
    cout << "Элементы из диапазона удалены" << endl;  
}
```

// Задание 5: Добавить среднее арифметическое (for\_each)

```
void task2_add_average(Stack& s) {  
    if (s.empty()) return;  
    Vector v = copy_stack_to_vector(s);  
    Pair sum(0, 0);  
    for_each(v.begin(), v.end(), [&sum](const Pair& val) { sum = sum + val; });  
    Pair avg = sum / v.size();  
    for_each(v.begin(), v.end(), [avg](Pair& val) { val = val + avg; });  
    s = copy_vector_to_stack(v);  
    cout << "Среднее арифметическое добавлено к каждому элементу" << endl;  
}
```

void run\_task2() {

```
    cout << "\n===== ЗАДАЧА 2: stack<Pair> =====" << endl;
```

```
    Stack s;
```

```
int n;

cout << "Введите количество элементов: ";

cin >> n;

s = make_stack(n);

cout << "Исходный стек: ";

print_stack(s);


int choice;

cout << "\nВыберите задание:\n";

cout << "1. Задание 3 - Добавить максимальный элемент (max_element)\n";

cout << "2. Задание 4 - Удалить элементы из диапазона (remove_if)\n";

cout << "3. Задание 5 - Добавить среднее арифметическое (for_each)\n";

cout << "Ваш выбор: ";

cin >> choice;


switch (choice) {

case 1:

    task2_add_max(s);

    cout << "Результат: ";

    print_stack(s);

    break;

case 2: {

    Pair key1, key2;

    cout << "Введите начало диапазона (Pair):\n";

    cin >> key1;

    cout << "Введите конец диапазона (Pair):\n";

    cin >> key2;

    task2_delete_range(s, key1, key2);

    cout << "Результат: ";

    print_stack(s);

    break;

}

case 3:

    task2_add_average(s);

    cout << "Результат: ";

    print_stack(s);
```

```

        break;
    default:
        cout << "Неверный выбор!" << endl;
    }
}

//=====

// ЗАДАЧА 3: set<Pair> с алгоритмами STL

//=====

typedef set<Pair> Set;

Set make_set(int n) {
    Set s;
    for (int i = 0; i < n; i++)
        s.insert(Pair(rand() % 50, rand() % 50));
    return s;
}

void print_set(const Set& s) {
    for (auto it = s.begin(); it != s.end(); ++it)
        cout << *it << " ";
    cout << endl;
}

// Задание 3: Найти максимальный элемент и добавить его
void task3_add_max(Set& s) {
    if (s.empty()) return;
    Pair max = *s.rbegin();
    s.insert(max);
    cout << "Максимальный элемент (" << max << ") добавлен" << endl;
}

// Задание 4: Удалить элементы из диапазона
void task3_delete_range(Set& s, Pair key1, Pair key2) {
    if (key1 > key2) swap(key1, key2);
    int count = 0;

```

```

    auto it = s.lower_bound(key1);
    while (it != s.end() && *it <= key2) {
        it = s.erase(it);
        count++;
    }
    cout << "Удалено элементов: " << count << endl;
}

// Задание 5: Добавить среднее арифметическое (for_each)
void task3_add_average(Set& s) {
    if (s.empty()) return;
    Pair sum(0, 0);
    for_each(s.begin(), s.end(), [&sum](const Pair& val) { sum = sum + val; });
    Pair avg = sum / s.size();

    Set newSet;
    for_each(s.begin(), s.end(), [avg, &newSet](const Pair& val) {
        newSet.insert(Pair(val.getFirst() + avg.getFirst(), val.getSecond() + avg.getSecond()));
    });
    s = newSet;
    cout << "Среднее арифметическое добавлено к каждому элементу" << endl;
}

void run_task3() {
    cout << "\n===== ЗАДАЧА 3: set<Pair> =====" << endl;
    Set s;
    int n;
    cout << "Введите количество элементов: ";
    cin >> n;
    s = make_set(n);
    cout << "Исходный set: ";
    print_set(s);

    int choice;
    cout << "\nВыберите задание:\n";
    cout << "1. Задание 3 - Добавить максимальный элемент (max_element/rbegin)\n";

```

```

cout << "2. Задание 4 - Удалить элементы из диапазона (lower_bound/erase)\n";
cout << "3. Задание 5 - Добавить среднее арифметическое (for_each)\n";
cout << "Ваш выбор: ";
cin >> choice;

switch (choice) {
case 1:
    task3_add_max(s);
    cout << "Результат: ";
    print_set(s);
    break;
case 2: {
    Pair key1, key2;

    cout << "Введите начало диапазона (Pair):\n";
    cin >> key1;

    cout << "Введите конец диапазона (Pair):\n";
    cin >> key2;

    task3_delete_range(s, key1, key2);

    cout << "Результат: ";
    print_set(s);
    break;
}
case 3:
    task3_add_average(s);
    cout << "Результат: ";
    print_set(s);
    break;
default:
    cout << "Неверный выбор!" << endl;
}
}

//=====

// ГЛАВНОЕ МЕНЮ

//=====

int main() {

```

```

setlocale(LC_ALL, "ru");
srand(time(0));

int choice;

do {
    cout << "\n=====\\n";
    cout << "1. Задача 1 - deque<Pair> (последовательный контейнер)\\n";
    cout << "2. Задача 2 - stack<Pair> (адаптер контейнера)\\n";
    cout << "3. Задача 3 - set<Pair> (ассоциативный контейнер)\\n";
    cout << "0. Выход\\n";
    cout << "=====\\n";
    cout << "Выберите задачу: ";
    cin >> choice;

    switch (choice) {
        case 1: run_task1(); break;
        case 2: run_task2(); break;
        case 3: run_task3(); break;
        case 0: cout << "Выход из программы..." << endl; break;
        default: cout << "Неверный выбор! Попробуйте снова." << endl;
    }

    if (choice != 0) {
        cout << "\nНажмите Enter для продолжения...";
        cin.ignore();
        cin.get();
    }
} while (choice != 0);

system("pause");
return 0;
}

```

## Результаты работы:

```
=====
1. Задача 1 - deque<Pair> (последовательный контейнер)
2. Задача 2 - stack<Pair> (адаптер контейнера)
3. Задача 3 - set<Pair> (ассоциативный контейнер)
0. Выход
=====
Выберите задачу: 1

===== ЗАДАЧА 1: deque<Pair> =====
Введите количество элементов: 6
Исходный deque: (9, 20) (26, 32) (13, 3) (45, 48) (34, 38) (39, 29)

Выберите задание:
1. Задание 3 - Добавить максимальный элемент в конец (max_element)
2. Задание 4 - Удалить элементы из диапазона (remove_if)
3. Задание 5 - Добавить среднее арифметическое (for_each)
Ваш выбор: 3
Среднее арифметическое добавлено к каждому элементу
Результат: (36.6667, 48.3333) (53.6667, 60.3333) (40.6667, 31.3333) (72.6667, 76.3333) (61.6667, 66.3333) (66.6667, 57.3333)

Нажмите Enter для продолжения...

=====
1. Задача 1 - deque<Pair> (последовательный контейнер)
2. Задача 2 - stack<Pair> (адаптер контейнера)
3. Задача 3 - set<Pair> (ассоциативный контейнер)
0. Выход
=====
Выберите задачу: 2

===== ЗАДАЧА 2: stack<Pair> =====
Введите количество элементов: 3
Элемент 1:
first? 2
second? 4
Элемент 2:
first? 5
second? 3
Элемент 3:
first?
5
second? 5
Исходный стек: Стек (сверху вниз): (2, 4) (5, 3) (5, 5)

Выберите задание:
1. Задание 3 - Добавить максимальный элемент (max_element)
2. Задание 4 - Удалить элементы из диапазона (remove_if)
3. Задание 5 - Добавить среднее арифметическое (for_each)
Ваш выбор: 2
Введите начало диапазона (Pair):
first? 2
second? 5
Введите конец диапазона (Pair):
first? 1
second? 4
Элементы из диапазона удалены
Результат: Стек (сверху вниз): (5, 3) (5, 5)

Нажмите Enter для продолжения...
```

```
=====
1. Задача 1 - deque<Pair> (последовательный контейнер)
2. Задача 2 - stack<Pair> (адаптер контейнера)
3. Задача 3 - set<Pair> (ассоциативный контейнер)
0. Выход
=====
Выберите задачу: 3

===== ЗАДАЧА 3: set<Pair> =====
Введите количество элементов: 4
Исходный set: (11, 15) (35, 17) (43, 26) (49, 29)

Выберите задание:
1. Задание 3 - Добавить максимальный элемент (max_element/rbegin)
2. Задание 4 - Удалить элементы из диапазона (lower_bound/erase)
3. Задание 5 - Добавить среднее арифметическое (for_each)
Ваш выбор: 3
Среднее арифметическое добавлено к каждому элементу
Результат: (45.5, 36.75) (69.5, 38.75) (77.5, 47.75) (83.5, 50.75)

Нажмите Enter для продолжения...

=====
1. Задача 1 - deque<Pair> (последовательный контейнер)
2. Задача 2 - stack<Pair> (адаптер контейнера)
3. Задача 3 - set<Pair> (ассоциативный контейнер)
0. Выход
=====
Выберите задачу: 0
Выход из программы...
Для продолжения нажмите любую клавишу . . .
```



# UML

